

Matrix Decompositions for AI/ML

Breaking Complex Matrices into Simpler Parts

LU Decomposition

Lower $\times$ Upper triangular

Eigenvalue Decomposition

Special directions & scaling

Singular Value Decomposition

Powerful & general-purpose





What is Matrix Decomposition?

A complicated matrix can be represented as a **product of simpler matrices**. Just like a complex machine is really a motor + gears + control system working together, a complex matrix can be broken into simpler parts that are much easier to understand and work with.

The Core Idea

$$\mathbf{A} = \mathbf{B} \times \mathbf{C}$$

One complex matrix becomes two simpler ones.

Why Decompose?

- Easier calculations
- Solve equations faster
- Understand data structure
- Numerical efficiency
- Power AI/ML algorithms

What is LU Decomposition?

Any suitable square matrix **A** can be written as the product of a **Lower triangular matrix L** and an **Upper triangular matrix U**. A lower triangular matrix has zeros above the diagonal; an upper triangular matrix has zeros below the diagonal. These simple shapes make calculations much faster.

The Formula

$$\mathbf{A} = \mathbf{L} \times \mathbf{U}$$

L = Lower Triangular

U = Upper Triangular

Matrix Structures

L (zeros above diagonal):

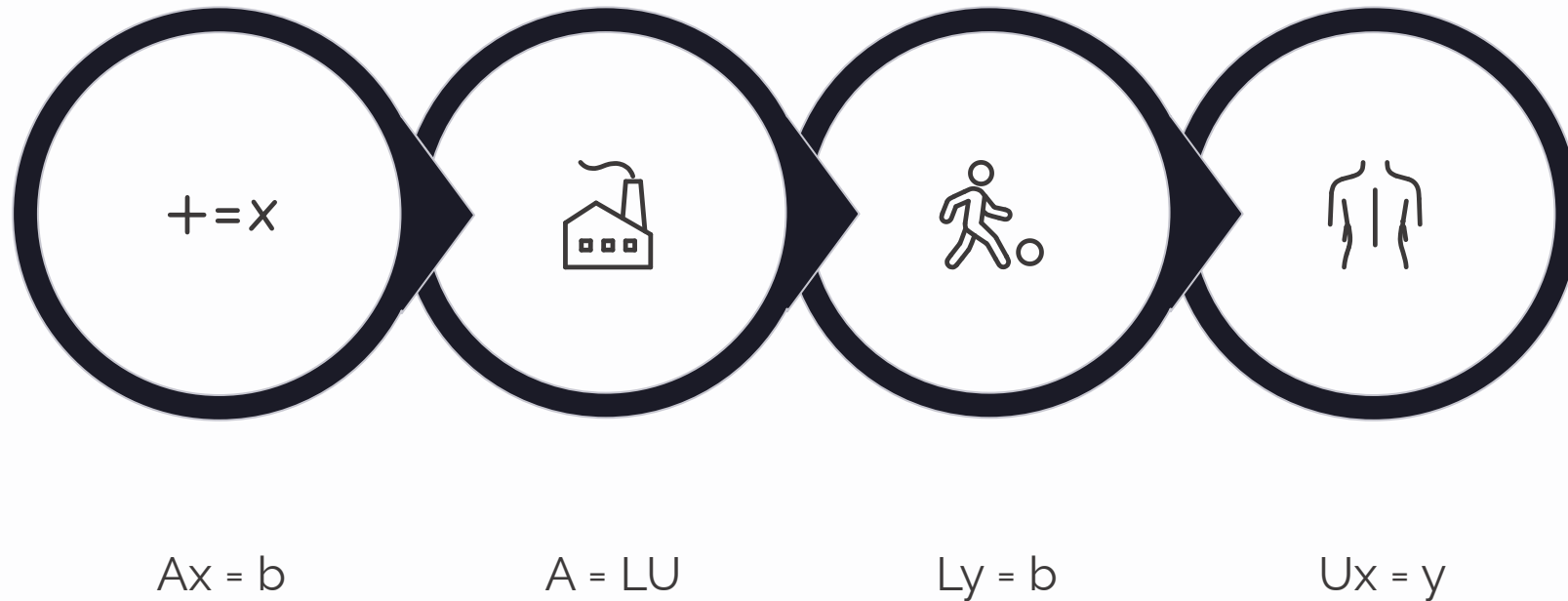
```
[ * 0 0 ]  
[ * * 0 ]  
[ * * * ]
```

U (zeros below diagonal):

```
[ * * * ]  
[ 0 * * ]  
[ 0 0 * ]
```

LU Decomposition — The Key Idea

Instead of solving one difficult problem $\mathbf{Ax} = \mathbf{b}$ directly, LU decomposition lets us solve **two simpler problems** in sequence. Think of it like a two-stage assembly line — each stage does one manageable job.



□ The magic: triangular systems are extremely easy to solve by simple substitution — no complex elimination needed!

STEP-BY-STEP EXAMPLE

LU Example — Finding U

Let's find the LU decomposition of a 2x2 matrix step by step. We apply **Gaussian elimination** to convert A into upper triangular form U, tracking the multipliers we use along the way.

Our Matrix A

$$A = \begin{bmatrix} 2 & 3 \\ 4 & 7 \end{bmatrix}$$

Goal: $A = LU$

Step 1 — Eliminate below pivot

The pivot is **2** (top-left). We want to eliminate the **4** below it.

Multiplier: $m = 4 \div 2 = 2$

Perform: $R2 \rightarrow R2 - 2 \times R1$

$$\begin{bmatrix} 2 & 3 \\ 4 & 7 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & 3 \\ 0 & 1 \end{bmatrix}$$

Therefore: $U = \begin{bmatrix} 2 & 3 \\ 0 & 1 \end{bmatrix}$

STEP-BY-STEP EXAMPLE

LU Example — Constructing L

The multiplier we used during elimination **goes directly into matrix L** at the position below the diagonal. The diagonal of L is always filled with **1s**. This elegant rule means we don't need any extra calculation to build L!

Building L from the multiplier

Our multiplier was $m = 2$

$$L = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}$$

Diagonal = 1s, below diagonal = multiplier

Verification: $L \times U = A$?

$$\begin{aligned} & \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & 3 \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 1 \times 2 + 0 \times 0 & 1 \times 3 + 0 \times 1 \\ 2 \times 2 + 1 \times 0 & 2 \times 3 + 1 \times 1 \end{bmatrix} \\ &= \begin{bmatrix} 2 & 3 \\ 4 & 7 \end{bmatrix} \checkmark \end{aligned}$$

✓ Result: $L = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}$ and $U = \begin{bmatrix} 2 & 3 \\ 0 & 1 \end{bmatrix}$. Verified: $L \times U = A$ ✓

Why LU Matters in AI/ML

LU decomposition is the backbone of **numerical linear algebra**. When training ML models, we often need to solve the same system $Ax = b$ many times with different right-hand sides. Decomposing $A = LU$ **once** makes all future solves fast and efficient.



Linear Systems

Solve $Ax = b$ efficiently using forward + back substitution.



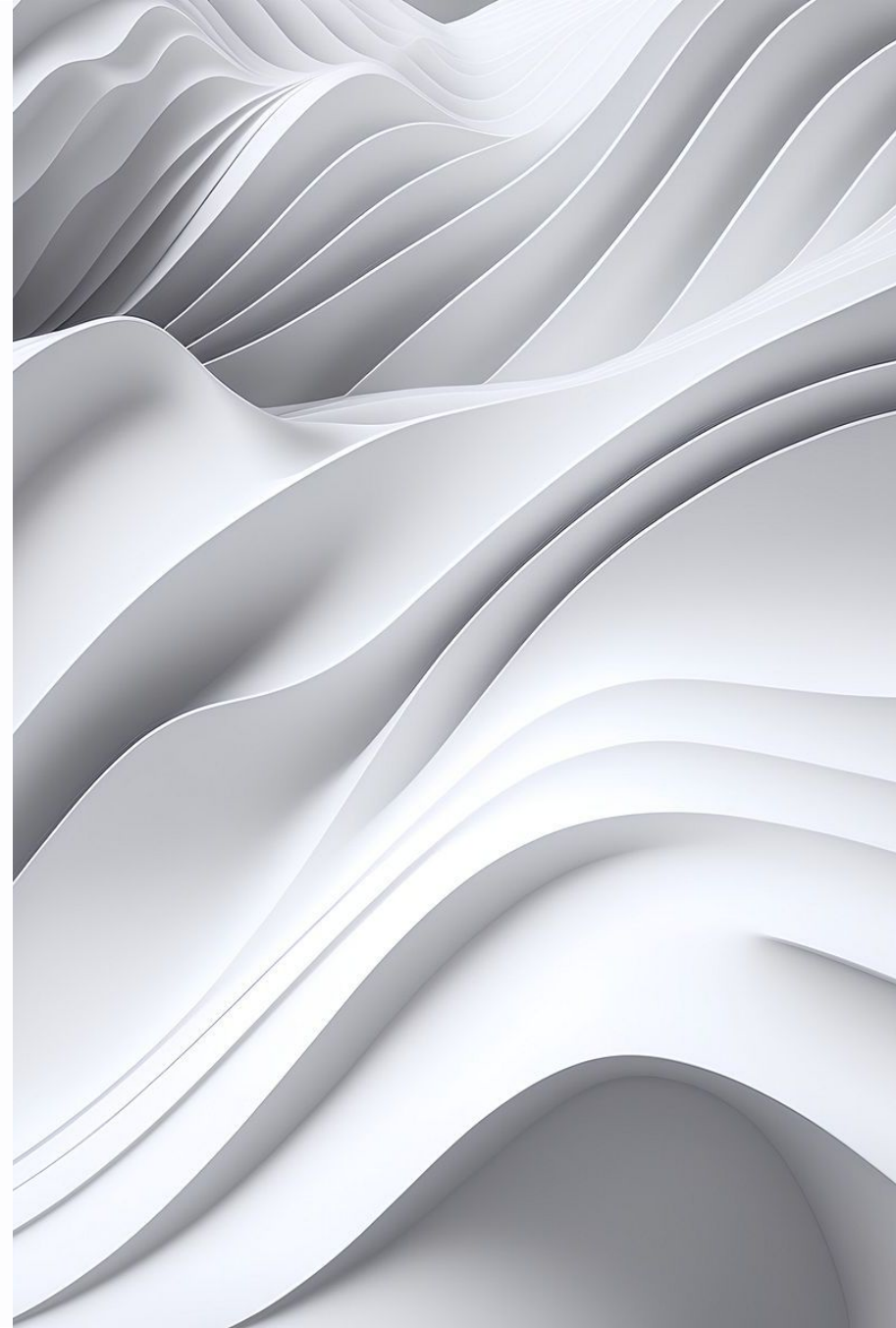
Parameter Estimation

Fitting model parameters in regression and optimization.



Numerical Efficiency

Decompose once, solve many times — avoids computing A^{-1} directly.



What is Eigenvalue Decomposition?

For a suitable square matrix, we can write $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$. This is built directly on the eigenvalue equation you already know: $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. Eigenvectors are special directions that *don't rotate* under the matrix transformation — they only stretch or shrink.

The Formula

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$$

$\mathbf{V}$ = matrix of eigenvectors (columns)

$\mathbf{\Lambda}$ = diagonal matrix of eigenvalues

$\mathbf{V}^{-1}$ = inverse of $\mathbf{V}$

The Eigenvalue Equation

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

$\mathbf{v}$ = eigenvector (special direction)

λ = eigenvalue (scaling factor)

- $\lambda = 3 \rightarrow$ stretched 3 times
- $\lambda = 0.5 \rightarrow$ shrunk to half
- $\lambda = -2 \rightarrow$ flips and doubles

STEP-BY-STEP EXAMPLE

Eigen Decomposition — Finding Eigenvalues

Let's work through a complete example. We use the **characteristic equation** $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$ to find eigenvalues. This is the same method you've already seen in class — we're just going one step further to decompose the matrix.

Our Matrix

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Step 1: Set up $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$

$$\begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix} = 0$$

Step 2: Expand the determinant

$$(2-\lambda)(2-\lambda) - (1)(1) = 0$$

$$(2-\lambda)^2 - 1 = 0$$

$$\lambda^2 - 4\lambda + 4 - 1 = 0$$

$$\lambda^2 - 4\lambda + 3 = 0$$

$$(\lambda - 3)(\lambda - 1) = 0$$

Eigenvalues:

$$\lambda_1 = 3 \text{ and } \lambda_2 = 1$$

Eigen Decomposition — Finding Eigenvectors

For each eigenvalue, we substitute back into $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$ and solve for the eigenvector. These eigenvectors become the **columns of $\mathbf{V}$** .

For $\lambda_1 = 3$:

$$\begin{aligned}(\mathbf{A} - 3\mathbf{I})\mathbf{v} &= \mathbf{0} \\ \begin{bmatrix} -1 & 1 \end{bmatrix} \mathbf{v} &= \mathbf{0} \\ \begin{bmatrix} 1 & -1 \end{bmatrix}\end{aligned}$$

$$\rightarrow \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

For $\lambda_2 = 1$:

$$\begin{aligned}(\mathbf{A} - \mathbf{I})\mathbf{v} &= \mathbf{0} \\ \begin{bmatrix} 1 & 1 \end{bmatrix} \mathbf{v} &= \mathbf{0} \\ \begin{bmatrix} 1 & 1 \end{bmatrix}\end{aligned}$$

$$\rightarrow \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$\mathbf{V}$ (eigenvectors as columns)

$$\mathbf{V} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

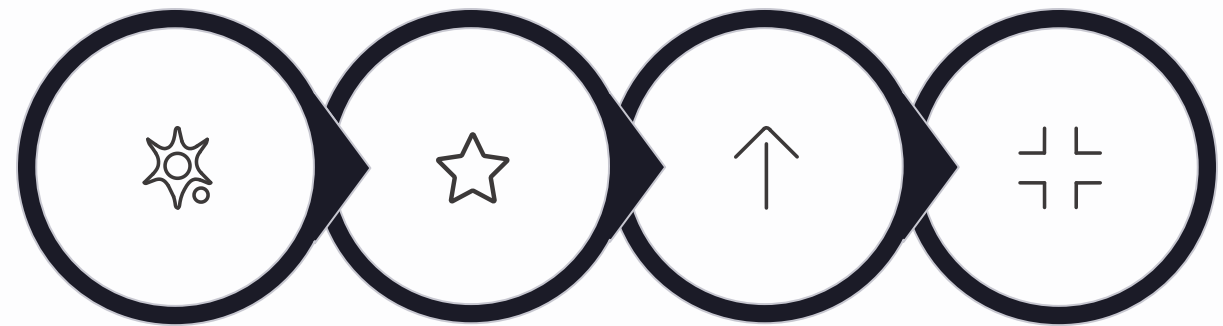
$\mathbf{\Lambda}$ (eigenvalues on diagonal)

$$\mathbf{\Lambda} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

Therefore: $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$ ✓

Eigen Decomposition in AI/ML — PCA

Eigenvalue decomposition is the mathematical foundation of **Principal Component Analysis (PCA)** — one of the most widely used techniques in machine learning for understanding and compressing high-dimensional data.



Covariance
Matrix

Eigen
Decompositio
n

Principal
Directions

Dimensionalit
y Reduction

Eigenvectors →

Important **directions** in
the data (principal axes)

Eigenvalues →

Variance (importance)
along each direction

Larger λ →

More variance = **more
information** in that
direction

What is Singular Value Decomposition (SVD)?

SVD is the **most powerful and general** matrix decomposition. Unlike eigenvalue decomposition (which works only on square matrices), SVD works on **any matrix** – rectangular or square. It decomposes A into three meaningful parts, each revealing something important about the data.

The Formula

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

$\mathbf{U}$ = Left singular vectors

$\mathbf{\Sigma}$ = Singular values (diagonal)

$\mathbf{V}^T$ = Right singular vectors (transposed)

SVD vs Eigen Decomposition

- **Eigen:** Square matrices only, uses eigenvectors
- **SVD:** Any $m \times n$ matrix, more general
- **SVD:** Always exists and is numerically stable
- **SVD:** Powers PCA, compression, recommendations

SVD Analogy — Image Compression

Think of SVD like **compressing a photo**. A large image is a huge matrix of pixel values. SVD finds the most important patterns (large singular values) and less important ones (small singular values). By keeping only the strongest patterns, we get a good approximation with much less storage.



Large Singular Value

= **Important** component, captures major structure

Small Singular Value

= **Less important**, often noise — can be discarded

SVD Step 1 — Calculate $A^T A$

The first step in computing SVD is to form the product $A^T A$. This is a square, symmetric matrix whose **eigenvalues will give us our singular values**. We always start here because $A^T A$ is easier to work with than A directly.

Our Matrix A

$$A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$$

Transpose A^T

$$A^T = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$$

(Here A is symmetric, so $A^T = A$)

Compute $A^T A$

$$\begin{aligned} A^T A &= \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \times \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \\ &= \begin{bmatrix} 3 \times 3 + 0 \times 0 & 3 \times 0 + 0 \times 2 \\ 0 \times 3 + 2 \times 0 & 0 \times 0 + 2 \times 2 \end{bmatrix} \\ &= \begin{bmatrix} 9 & 0 \\ 0 & 4 \end{bmatrix} \end{aligned}$$

$A^T A$ is always **square and symmetric** — its eigenvalues are always ≥ 0 .

SVD Steps 2 & 3 — Eigenvalues $\rightarrow$ Singular Values

We find eigenvalues of $A^T A$, then take their **square roots** to get the singular values. This is the key relationship connecting SVD to eigenvalue decomposition.

Step 2: Eigenvalues of $A^T A$

$$\det(A^T A - \lambda I) = 0$$

$$\begin{vmatrix} 9-\lambda & 0 \\ 0 & 4-\lambda \end{vmatrix} = 0$$

$$(9-\lambda)(4-\lambda) = 0$$

$$\lambda_1 = 9, \lambda_2 = 4$$

Step 3: Singular Values $\sigma_i = \sqrt{\lambda_i}$

$$\sigma_1 = \sqrt{\lambda_1} = \sqrt{9} = 3$$

$$\sigma_2 = \sqrt{\lambda_2} = \sqrt{4} = 2$$

Therefore:

$$\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$$

Singular values are always non-negative and arranged in decreasing order.

📌 Key insight: $\sigma_i = \sqrt{(\text{eigenvalue of } A^T A)}$. Larger σ = more important direction!

SVD Steps 4 & 5 — Finding V and U

The columns of $\mathbf{V}$ are eigenvectors of $A^T A$. Once we have $\mathbf{V}$, we compute $\mathbf{U}$ using the formula $u_i = Av_i / \sigma_i$. This connects all three components together elegantly.

Step 4: $\mathbf{V}$ from eigenvectors of $A^T A$

$$\text{For } \lambda_1 = 9: \mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\text{For } \lambda_2 = 4: \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\mathbf{V} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \mathbf{V}^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Step 5: $\mathbf{U}$ using $u_i = Av_i / \sigma_i$

$$\begin{aligned} A\mathbf{v}_1 &= \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix} \\ u_1 &= \begin{bmatrix} 3 \\ 0 \end{bmatrix} / 3 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \end{aligned}$$

$$\begin{aligned} A\mathbf{v}_2 &= \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \times \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix} \\ u_2 &= \begin{bmatrix} 0 \\ 2 \end{bmatrix} / 2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{aligned}$$

$$\mathbf{U} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Complete SVD — Putting It All Together

Let's bring all three components together and verify that $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. Seeing the full picture confirms that our decomposition is correct and makes the formula concrete.

The Three Components

$$\mathbf{U} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \leftarrow \text{Left singular vectors}$$

$$\mathbf{\Sigma} = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \leftarrow \text{Singular values}$$

$$\mathbf{V}^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \leftarrow \text{Right singular vectors}$$

Verification: $\mathbf{U} \times \mathbf{\Sigma} \times \mathbf{V}^T$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \times \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \times \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \times \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} = \mathbf{A} \quad \checkmark$$

✔ $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ verified! Each component plays a distinct role: $\mathbf{V}$ rotates input space, $\mathbf{\Sigma}$ scales, $\mathbf{U}$ rotates output space.

Why SVD is Essential in AI/ML

SVD is arguably the single most useful matrix decomposition in machine learning. Its ability to reveal hidden structure in **any** matrix — regardless of shape — makes it a universal tool across many AI domains.



PCA & Dimensionality Reduction

SVD on the data matrix directly gives principal components — no covariance matrix needed.



Image Compression

Keep top-K singular values to reconstruct images with much less storage.



Recommendation Systems

Netflix, Amazon — SVD finds hidden user-item preference patterns in sparse matrices.



Noise Reduction & Low-Rank Approximation

Discard small singular values to remove noise and represent data compactly.



COMPARISON

LU vs Eigenvalue vs SVD — At a Glance

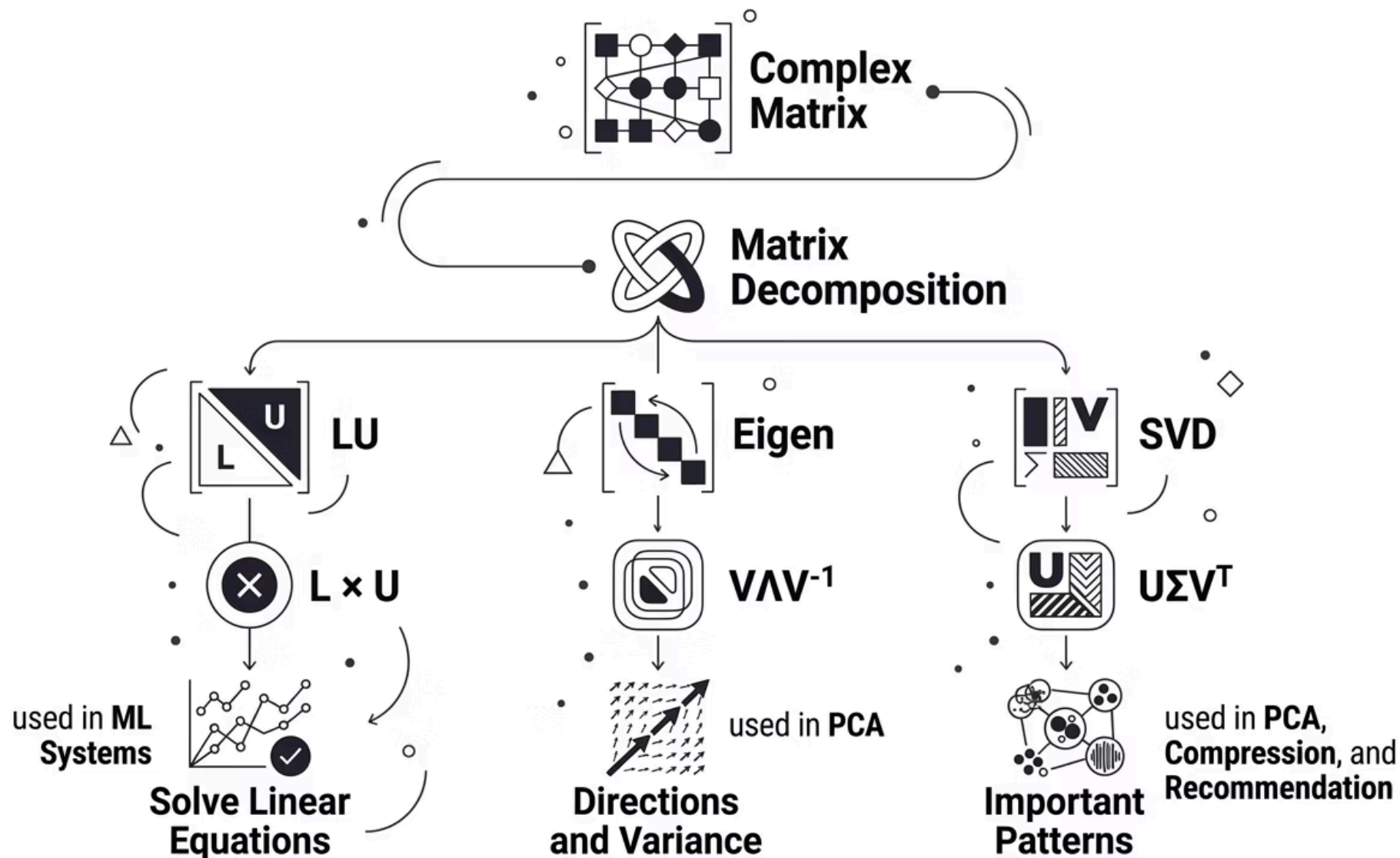
Each decomposition has a distinct purpose. Choosing the right one depends on **what you need to do** — solve equations, understand directions, or extract patterns from any matrix.

Decomposition	Formula	Main Idea	AI/ML Use
LU	$A = LU$	Lower + Upper triangular — two easy solves	Solving linear systems, parameter estimation
Eigen	$A = V\Lambda V^{-1}$	Special directions + scaling factors	PCA via covariance matrix, graph analysis
SVD	$A = U\Sigma V^T$	Singular directions + strengths — works on any matrix	PCA, compression, recommendation systems

📌 Rule of thumb: LU → computation, Eigen → understanding directions, SVD → powerful general-purpose factorization for data.

The Big Picture — Matrix Decompositions Unified

Every decomposition starts from the same challenge: a complex matrix. Depending on your goal, you choose the right tool. Together, these three decompositions power a huge portion of modern machine learning.



Student Revision — Key Formulas

Before we look at practice problems, make sure you can recall these three formulas **from memory** and explain what each letter means. That's the real test of understanding!

LU Decomposition

$$\mathbf{A} = \mathbf{L} \times \mathbf{U}$$

L = lower triangular (1s on diagonal, multipliers below) | U = upper triangular (from Gaussian elimination)

Eigenvalue Decomposition

$$\mathbf{A} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^{-1}$$

V = eigenvectors as columns | $\mathbf{\Lambda}$ = eigenvalues on diagonal | Equation: $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$

Singular Value Decomposition

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

U = left singular vectors | Σ = singular values ($\sigma = \sqrt{\lambda}$ of $\mathbf{A}^T\mathbf{A}$) | V = right singular vectors

01

What does matrix decomposition mean and why do we do it?

02

What are L and U — how is each one structured?

03

What do eigenvectors and eigenvalues represent geometrically?

04

How are singular values obtained from $\mathbf{A}^T\mathbf{A}$? What does a large singular value mean?

Practice Problems — Try These!

Work through these problems on your own. For each one, write out every step — don't jump to the answer. The process matters as much as the result.

LU Decomposition

1. Find the LU decomposition of:

$$A = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}$$

2. Find the LU decomposition of:

$$A = \begin{bmatrix} 3 & 2 \\ 6 & 5 \end{bmatrix}$$

Eigenvalue Problems

3. Find eigenvalues and eigenvectors of:

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$$

4. Find eigenvalues of:

$$A = \begin{bmatrix} 5 & 2 \\ 2 & 5 \end{bmatrix}$$

SVD Problems

5. Find the full SVD of:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

Hint: Start by computing $A^T A$, then find eigenvalues, singular values, V , and U .

6. A matrix has singular values $\sigma_1 = 5$, $\sigma_2 = 2$, $\sigma_3 = 0.5$.

- Which component captures the most important information?
- Which component could you drop in a low-rank approximation?
- What percentage of "energy" does σ_1 represent out of total?

 Tip for Problem 6: Energy $\propto \sigma_i^2$. Total = $25 + 4 + 0.25 = 29.25$.
First component = $25/29.25 \approx 85\%$!